BSDS 100: Final Project Report

Machine Learning: Classification

Arnav Khetan

8th December 2023

This is my BSDS 100: Final Project Report so I have put in content from both my code and slides that I used for the presentation on December 5th.

Thank you sir for the class, I had a lot of fun during the classes and learned a lot.

I chose to do the Machine Learning Teaching Project for the final project and in that I chose to do the Technique/Field of Classification. I worked alone on this project so I hope that doesn't negatively affect my scores in the teamwork aspects of the rubric. I have not included many figures (graphs) because I could not see any use cases of them to aid in my report nor have included multiple features of graphs in them (like legends and axis labels etc.).

Intro to Machine Learning (ML)

- Machine Learning is an area of Artificial Intelligence (AI) that allows AI models to learn and test things by themselves with minimal amounts of human intervention during both the learning and testing processes.
- It is primarily done using a training and testing data set. The training data set is used to teach the AI and the AI runs through it over multiple epochs (times) while the testing data set is used for testing.
- The training data set is a massive data set consisting of data from variety of different categories/classes while the testing data set is a small data set mostly focusing on a particular category/class. The size and focus of the testing data set depends on the accuracy of prediction of the model.
- This training and testing typically happens without any predefined equations so the AI is not given any prior information about the data sets or any sort of information that it could use as a basis for its learning process.
- ML is divided into 2 types: Supervised Learning & Unsupervised Learning

Supervised Learning vs Unsupervised Learning

- Both supervised and unsupervised learning use training and testing data set to train the AI models but supervised learning uses data that have all the categories/classes sorted (prior to been given to the AI) with clearly defined labels.
- Labeled data means that the data will contain both the input and output or question and answer for each data-point/observation. So, the AI will know the corresponding label for each data-point (the labels are there in both the training data set and the testing data set).
- Unsupervised Learning data sets don't contain labeled data.
- Classification is a form of Supervised Learning.

Classification

- Classification is defined as predicting a category or class for data-points/observations within a data set.
- The output is generated as a discrete output (exact values).

Types of Classification

- Binary Classification - Simplest form. 1s and 0s, TRUE and FALSE, Yes and No. Classifying between 2 categories.
- Multi-class Classification - Classifying data into 3 or more categories but each observation can be only be in one of the categories.
- Multi-label Classification - Classifying data into 3 or more categories but each observation can simultaneously be in 2 or more categories (isn't limited to only one classifier).

Classification Real World Examples

Email Filtering

- Binary Classification.
- Can be used to filter incoming emails for whether they are spam or not. This will primarily be done with the data being in strings containing keywords of things like common advertising terms, known company email spammers, etc.

Distracted Drivers

- Multi-class/Multi-label Classification.
- Countless people every year get into car accidents due to being distracted such as by looking at the phone or by falling asleep.
- AI Classification model can be trained with images of such distractions and learn to classify each distraction into a separate category.
- Depending on the category that a particular distraction gets classified into, the model can perform various actions to alert the driver.
- For example, if they are falling asleep - a loud sound could be played to wake up them up. Looking at the phone - play a voice message to alert the driver.

Challenges in Classification

- General Challenge: Since classification uses labeled data, any changes done to the data such as editing of the labels makes it hard for the AI model to adapt to this change. It has learned using the labels the entire time so a simple change in the labels will cause errors in the accuracy of the model (this wouldn't have been a problem for the unsupervised learning AI models as they don't use labeled data).
- Challenge for Me: Accuracy is not always what it seems. A relatively high accuracy model may still have errors and problems with the other types of accuracy measurements. Will be explained in further detail later on in the report.

Logistic Regression

- Primarily used in Binary Classification. Simplest ML technique as it's easy to interpret and understand.
- Predicts the probability of belonging to a particular category (discrete outcome).

The Data Set

- The data set contains a list of 5,324 people who attempted to climb to Mount Everest. It is a combination of 5,324 rows and 68 columns.
- I don't have any particular interest or reason for choosing this data set. I just thought that it was pretty intriguing to think about all the various factors and things that go into planning a mountain climb and how much of it is skill and/or luck dependent.
- The main dependent variable used with any analysis that can be done on this data set is a column called "success". It is a discrete and binary variable as it only has values of 1 or 0. 1: Success, 0: Failure (either by quitting or losing life).
- Using `read.csv()` to read in the data into R and saving that in a variable called "data".

```
data = read.csv("C:\\Users\\arnav\\Documents\\USF\\Coursework\\BSDS100_R\\BSDS100_FinalProjectMtEverestI

head(data) #Prints the first 6 rows of the data set.
```

```
##                     fname         lname                                name
## 1          A. Wahab Faraj    Al Qassimi             A. Wahab Faraj Al Qassimi
## 2           Aaron Charles        Newman                  Aaron Charles Newman
## 3            Aaron Fisher        Mainer                   Aaron Fisher Mainer
## 4                   Aasif          Jaan                           Aasif Jaan
## 5            Abdul Jabbar        Bhatti                    Abdul Jabbar Bhatti
## 6 Abdul Nassar Peringodan Ayyappanthody Abdul Nassar Peringodan Ayyappanthody
##    citizen sex age  yob           status leader deputy sherpa tibetan msolo
## 1  Bahrain man  30 1976          Climber      0      0      0       0     0
## 2      USA man  47 1971          Climber      0      0      0       0     0
## 3      USA man  31 1981 Assistant Leader      0      1      0       0     0
## 4    India man  27 1991          Climber      0      0      0       0     0
## 5 Pakistan man  59 1957          Climber      0      0      0       0     0
## 6    India man  42 1976          Climber      0      0      0       0     0
##   mtraverse mski mparapente mspeed abovebc success o2used o2none smtcnt
## 1         0    0          0      0       0       0      0      1      0
## 2         0    0          0      0       1       0      0      1      0
## 3         0    0          0      0       1       1      1      0      1
## 4         0    0          0      0       1       1      1      0      1
## 5         0    0          0      0       1       1      1      0      1
## 6         0    0          0      0       1       1      1      0      1
##   msmtdate1 msmtdate2 msmtdate3 claimed disputed inj death deathdate deathtype
## 1         -         -         -       0        0   0     0         -         0
## 2  04/22/19         -         -       0        0   0     0         -         0
## 3  05/20/13         -         -       0        0   0     0         -         0
## 4  05/20/18         -         -       0        0   0     0         -         0
## 5  05/21/17         -         -       0        0   1     0         -         0
## 6  05/16/19         -         -       0        0   0     0         -         0
```

```
##   deathclass deathhgtm ams weather mperhighpt msmtbid msmtterm yearseas peakid
## 1          0         0   0       0          0       1        6 2006 Spr   EVER
## 2          0         0   0       0       6100       1        7 2019 Spr   EVER
## 3          0         0   0       0       8850       5        1 2013 Spr   EVER
## 4          0         0   0       0       8850       5        1 2018 Spr   EVER
## 5          0         0   0       0       8850       5        1 2017 Spr   EVER
## 6          0         0   0       0       8850       5        1 2019 Spr   EVER
##         expid membid  host comrte stdrte         route1 route2 myear mseason
## 1 EVER-061-90      2 China      1      1 N Col-NE Ridge   <NA>  2006  spring
## 2 EVER-191-10     22 Nepal      1      1 S Col-SE Ridge   <NA>  2019  spring
## 3 EVER-131-81      4 Nepal      1      1 S Col-SE Ridge   <NA>  2013  spring
## 4 EVER-181-03      6 Nepal      1      1 S Col-SE Ridge   <NA>  2018  spring
## 5 EVER-171-04      4 Nepal      1      1 S Col-SE Ridge   <NA>  2017  spring
## 6 EVER-191-18      2 Nepal      1      1 S Col-SE Ridge   <NA>  2019  spring
##   maxatt att1st peakexps peakabvbc peaksmts highexps highabvbc highsmts allexps
## 1      0   TRUE        0         0        0        0         0        0       0
## 2      0   TRUE        0         0        0        0         0        0       0
## 3      0   TRUE        0         0        0        0         0        0       0
## 4      0   TRUE        0         0        0        0         0        0       0
## 5      0   TRUE        0         0        0        0         0        0       0
## 6      0   TRUE        0         0        0        0         0        0       0
##   allabvbc allsmts maxhighpt maxpeakid smry fullsmry          commerc ageGroup2
## 1        0       0         0      <NA> <NA>     <NA> commercial route        NA
## 2        0       0         0      <NA> <NA>     <NA> commercial route        NA
## 3        0       0         0      <NA> <NA>     <NA> commercial route        NA
## 4        0       0         0      <NA> <NA>     <NA> commercial route        NA
## 5        0       0         0      <NA> <NA>     <NA> commercial route        NA
## 6        0       0         0      <NA> <NA>     <NA> commercial route        NA
##   ageGroup3    era
## 1     young recent
## 2    midage recent
## 3     young recent
## 4     young recent
## 5    midage recent
## 6    midage recent
```

```r
ncol(data) #Gives the total number of columns in the data set.
```

```
## [1] 68
```

```r
nrow(data) #Gives the total number of rows in the data set.
```

```
## [1] 5324
```

- I decided to use the built-in logistic regression approach in R instead of dividing my data into a training data set and a testing data set.
- Next, I decided to use 2 variables as my independent variables for the logistic regression.
- "o2used" is the "yes" or "no" binary variable for whether supplemental oxygen such as through gas masks or gas tanks were used or not while climbing Mount Everest (discrete variable).

- "mspeed" is the measure of the speed of the climbers during the climb (continuous variable). Further analysis on this column (done after the presentation on Dec 5th) showed that the entire column is filled with 0s... so the logistic regression was always only dependent on the "o2used" variable. Nevertheless, I will still be providing my code for when "mspeed" was used because a lot of findings can still be made with that.

The Code

- `glm()` is the function for Generalized Linear Model which is basically the same as `lm()` (Linear Model) except for the `family` argument. This argument is crucial as it allows us to tell R that we want to do logistic regression by filling in "binomial". "Binomial" is a type of mathematical distribution that refers to the number of successes in a series of independent trials (each trial has 2 outcomes) showing that it is relevant for logistic regression.

```
model = glm(success ~ mspeed + o2used, data = data, family = "binomial")
summary(model)
```

```
##
## Call:
## glm(formula = success ~ mspeed + o2used, family = "binomial",
##     data = data)
##
## Coefficients: (1 not defined because of singularities)
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -3.5264     0.1449  -24.33   <2e-16 ***
## mspeed           NA         NA      NA       NA
## o2used        4.6762     0.1501   31.16   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 7368.3  on 5323  degrees of freedom
## Residual deviance: 4426.9  on 5322  degrees of freedom
## AIC: 4430.9
##
## Number of Fisher Scoring iterations: 6
```

- I have not used the `summary(model)` results for my findings due to the "mspeed" being NA as it has all 0s for values, instead I have used something called Classification Metrics which will be seen later on in the report.

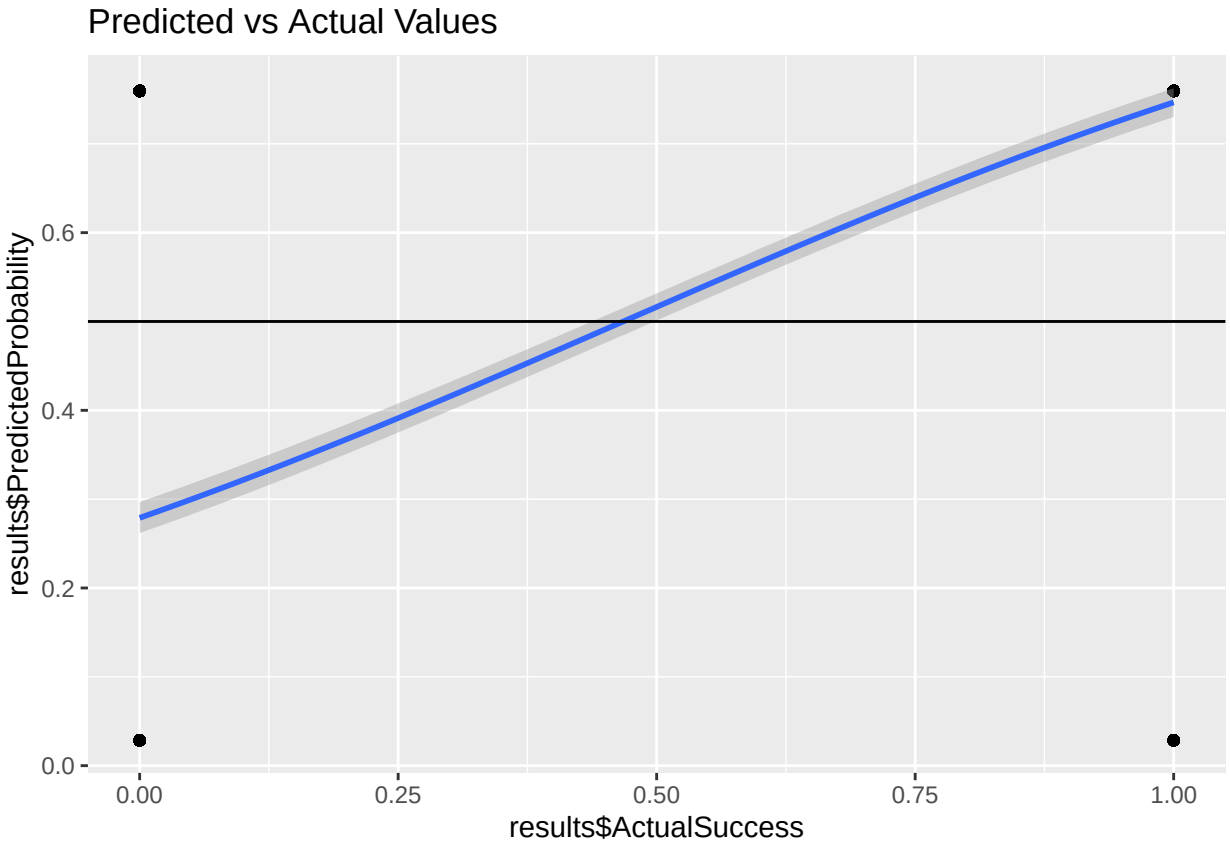
```
predicted_prob = predict(model, type = "response")
results = data.frame(ActualSuccess = data$success, PredictedProbability = predicted_prob)

head(results)
```

```
##   ActualSuccess PredictedProbability
## 1             0           0.02857143
## 2             0           0.02857143
## 3             1           0.75949016
```

```
## 4             1           0.75949016
## 5             1           0.75949016
## 6             1           0.75949016
```

```
library(ggplot2)
ggplot(results, aes(x = results$ActualSuccess, y = results$PredictedProbability)) +
  geom_point() +
  labs(title = "Predicted vs Actual Values") +
  geom_smooth(method = "glm", method.args = list(family = "binomial")) +
  geom_hline(yintercept = 0.5)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



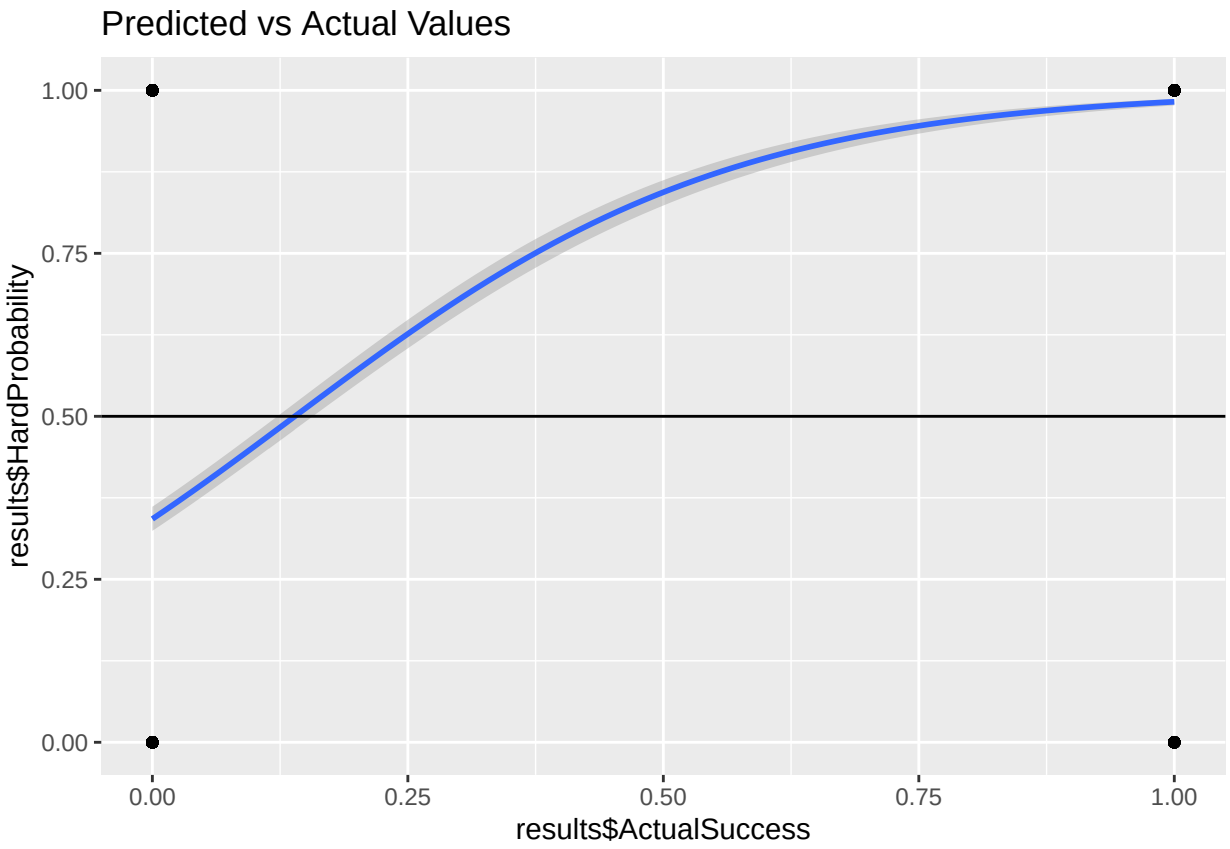
- The predicted probability values for the logistic regression have been made and I have created a graph to show them in comparison to the actual values. Even though, there are only 4 points on the graph, it is still easy to recognize that it is not possible to use these predicted probability values as they have too many decimal values.
- Therefore, have created a threshold. If the predicted probability values are greater than 0.5 than they shall be considered as a (1) success and if they are less than 0.5 then they shall be considered as a 0 (failure). All of these new values will be stored in a new column of the "results" data-frame called "HardProbability".

```r
for(i in 1:nrow(results)){
  if(results$PredictedProbability[i] > 0.5){
    results$HardProbability[i] = 1
  }
  else if(results$PredictedProbability[i] < 0.5){
    results$HardProbability[i] = 0
  }
}

head(results)
```

```
##   ActualSuccess PredictedProbability HardProbability
## 1             0           0.02857143               0
## 2             0           0.02857143               0
## 3             1           0.75949016               1
## 4             1           0.75949016               1
## 5             1           0.75949016               1
## 6             1           0.75949016               1
```

```r
ggplot(results, aes(x = results$ActualSuccess, y = results$HardProbability)) +
  geom_point() +
  labs(title = "Predicted vs Actual Values") +
  geom_smooth(method = "glm", method.args = list(family = "binomial")) +
  geom_hline(yintercept = 0.5)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



- Now with the "HardProbability" values, even though there are still only 4 points on the graph (seemingly the same as well due to the axis values), the regression line is now curved. This is a good change as this is logistic regression so the line is supposed to be curved and not linear. It also tells us that now some relationships can be formulated between the actual values and predicted probabilities.

Confusion Matrix

- I proceeded to create a confusion matrix which is basically a table that is used to show the performance of a model by comparing the predicted probability values to the actual values.
- I first installed and imported the "caret" library and then created the confusion matrix in a variable called "matrix".
- A confusion matrix looks like this:

- I created this table using Microsoft Excel.

```
library(caret)
```

```
## Loading required package: lattice
```

Confusion Matrix		Actual Values	
		Actually Positive (1)	Actually Negative (0)
Predicted Probability Values	Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
	Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

Figure 1: Confusion Matrix

```r
matrix = confusionMatrix(data = as.factor(results$HardProbability),
                         reference = as.factor(results$ActualSuccess))

matrix
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1666   49
##          1  868 2741
##
##                Accuracy : 0.8278
##                  95% CI : (0.8173, 0.8378)
##     No Information Rate : 0.524
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.6495
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.6575
##             Specificity : 0.9824
##          Pos Pred Value : 0.9714
##          Neg Pred Value : 0.7595
##              Prevalence : 0.4760
##          Detection Rate : 0.3129
##    Detection Prevalence : 0.3221
##       Balanced Accuracy : 0.8199
##
##        'Positive' Class : 0
##
```

- Running the "matrix" variable gives many values, which are called as "Classification Metrics".

Classification Metrics

- Used to show the performance of the AI model in terms of numbers/percentages/graphs and help to understand whether the model has done its job correctly or not. It is also where the challenge in classification as mentioned previously arises.
- The Classification Metrics which I think to be the most important and most relevant to my model are: Accuracy, Sensitivity (Recall), Specificity, ROC Curve.

Accuracy

- Firstly, the Accuracy provided by the confusion matrix is approximately 83% (rounded up) which is a relatively high percentage considering that the AI model is only being made using logistic regression and 1 independent variable (o2used) but here is where the challenge arises.
- Even though this accuracy is fairly high, the other classification metrics don't reflect the same results as the accuracy does.

Sensitivity

- Sensitivity, also known as Recall, is the value for "out of all the actual positives, how many were predicted to be positives" (known as True Positives (TPs) based on the confusion matrix table). This was approximately 66% (rounded up). So, put simply it is when the actual value is 1 and the predicted value is also 1. It is the most important metric for us because we need to make sure that the AI model is able to correctly predict 1 for when the actual value is also 1.

Specificity

- Similarly, Specificity which is the value for "out of all the actual negatives, how many were predicted to be negatives" (known as True Negatives (TNs) based on the confusion matrix table) was approximately 98% (rounded down). This is when the actual value is 0 and predicted value is also 0.
- A reason for this inaccuracy/challenge in the model is due to the number of people who were actual positives is itself very less. Out of all the 5,324 climbers, very few of these people were actual positives who successfully climbed Mount Everest. So, because I only had 1 independent variable, even if I had 2 independent variables, this inaccuracy would still be there because the number of people who got 1 in success is itself very low. Perhaps, if multiple independent variables had been used then the sensitivity could have been higher.

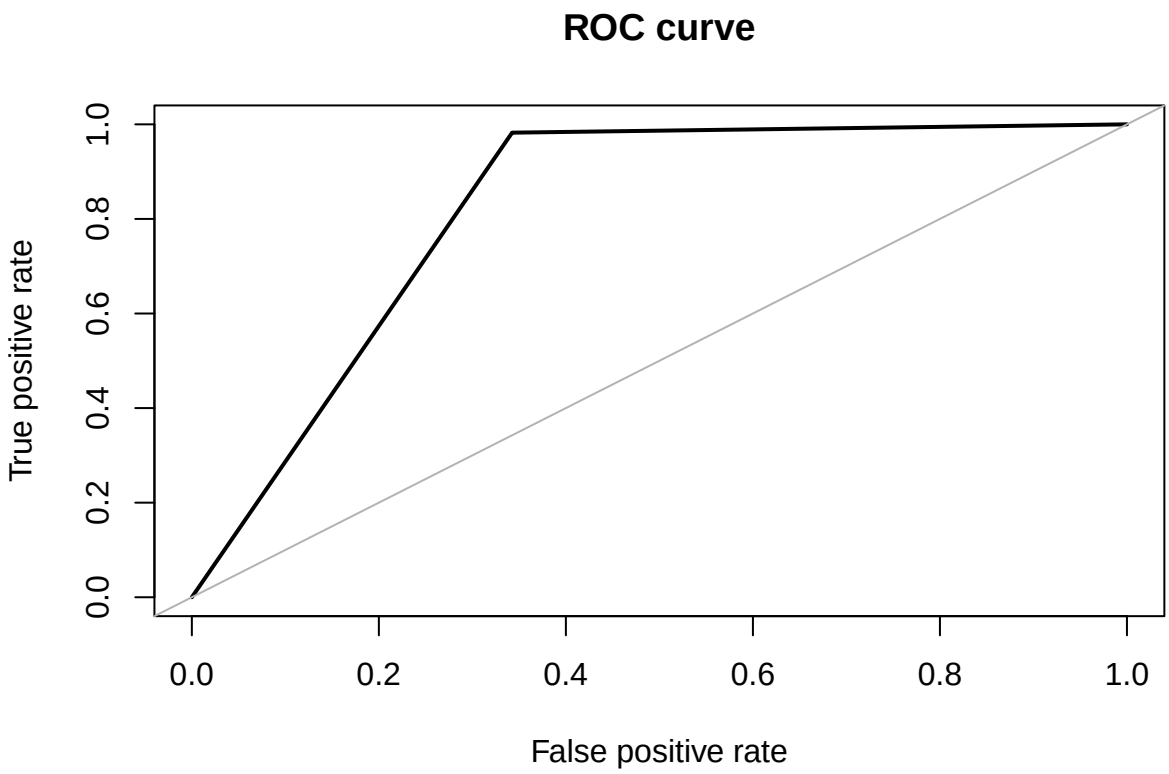
ROC Curve

- The last metric is the ROC Curve which is a graphical representation of the confusion matrix used to show the performance of a binary classifier model (which is what this is as it uses logistic regression) at varying thresholds. It also helps to see if the accuracy is accurate or not.
- I first installed and imported the "ROSE" library and then created the ROC Curve.

```
library(ROSE)
```

```
## Loaded ROSE 0.0-4
```

```
roc.curve(results$ActualSuccess, results$HardProbability, n.thresholds = 1000)
```




```
## Area under the curve (AUC): 0.820
```

- Normally, if the other classification metrics reflected the high percentage of the accuracy then the ROC Curve would be more curved and have more irregularities in its shape and occupy the top left region of the graph but since it is not, the curve is like this. It is a straight diagonal line which then straightens out later.
- The completely straight dull line is a representation of random guesses and so if any portion of the curve went below that, then the model is completely inaccurate; however no part of the graph is below that.
- The Area under the ROC Curve is also what is used to define the model's performance. This is not the same as "Accuracy", notice how the output from the ROC Curve says "Area under the curve (AUC): 0.820, which is 82% (rounded down).

Real World Example of this AI Model

- This model, using the same data set, could be used as a way to help people decide whether they should climb Mount Everest or not:
- People can input in their statistics for each independent variable such as their speed or if they will be using any supplemental oxygen.
- The accuracy of the model is 82% so a potential climber who is thinking of climbing to Mount Everest could make use of it.

- Since the sensitivity is only 66%, there is not much of any proper guarantee that the model's prediction based on their input will be accurate. So, even if the model predicts a "1", it is best to not proceed with the climb.
- However, since specificity is at 98%, if someone's success gets predicted as "0" then that person will not do the climb.

Conclusion

- I worked alone on this project and report so everything in this is done by me. While I did not use proper training and testing data sets and created the model with the built-in version of logistic regression in R, I think this can still be used by other DS students.
- This is mainly with reference to the Classification Metrics because all sorts of AI models will contain actual and predicted values of things. So those can be used to create a confusion matrix and then the classification metrics can be used to help understand the accuracy of the model (even if they are not using classification) and help in their respective conclusions and findings.
- This model helped me to understand that "Accuracy" isn't everything. There are a variety of different things that must be taken into consideration about an AI model before deciding that it is full-proof and accurate, not just the overall accuracy.
- Overall, I have learnt a lot from this project and hope to use some of my learnings in the future.